Voice-to-Website AI: Final Project Plan

One shared plan for scope, architecture, ownership, timeline, budget and risks. Version 1.0, 17 Oct 2026. Companion image: Agentic_Architecture_v2.png.

1. The project in one paragraph

Voice-to-Website AI is a SaaS platform where a non-technical shop owner describes a website by voice or text, answers a few clarifying questions, and receives a real, buildable website project (source code, not a template fill). A team of AI agents generates the code, builds and tests it in an isolated sandbox, and commits every version to GitHub. The owner previews the site and keeps requesting changes in chat. Production deployment is an optional, human-assisted paid service: the owner requests it, a developer from our team is assigned, they chat in a dedicated room, the owner pays through the platform, the developer deploys, and the platform pays developers monthly after taking a commission.

Three rules everyone must remember

1. Code controls the flow. A LangGraph state graph decides routing; LLMs never choose the next step freely.
2. Verify with tools, not opinions. A result is done only when the sandbox build and Playwright checks pass.
3. Clarify before generating. No code is written until the user confirms a requirements summary.

2. Locked decisions

Area	Decision
Product	Voice or text in, real website source code out, preview, optional human deployment
Input	Text and voice. Whisper-class ASR. The transcript is always shown back for confirmation
Stacks (MVP)	Two pinned profiles: vite-react-tailwind (primary) and static-html. Next.js profiles are future work

Area	Decision
Orchestration	LangGraph state graph, Postgres checkpointer, interrupt() for user answers, typed Pydantic handoffs between agents
Agents	5 LLM agents in the MVP: Clarifier, Planner, Coder, Editor, Fixer. Router and Locator start as rules/code and become LLM agents only if evals show a need. The full design has 7 agents plus an optional vision judge
Sandbox	E2B Hobby tier or local Docker. 2 vCPU, 2 GB RAM, 5 minute timeout, no secrets, egress limited to the npm registry. RAM setting is tested in week 1
Versioning	GitHub is the source of truth. A code-only Committer node calls the official GitHub MCP server (github/github-mcp-server). One commit and tag vN per verified version. Failed attempts go to a draft branch, never main. Rollback means revert or checkout of an older tag
Database	PostgreSQL + pgvector: messages, requirements, versions (with commit SHA), file manifest and embeddings, decisions, runs, plus marketplace tables
Object storage	S3-compatible bucket for audio, screenshots and zips (file blobs are no longer stored there)
Preview	Static build output served at a preview URL (from the sandbox or a static host)
Deployment	Human-in-the-loop: ticket, assigned developer, dedicated chat, quote, payment, developer deploys, user confirms
Payments	Internal ledger plus a sandbox/mock gateway. Stripe does not support Pakistan-registered merchants, so a local gateway (for example Safepay) is future work. Payouts are a monthly report; real bank transfers stay manual

Area	Decision
Subscriptions	Free and Pro plans with generation and edit quotas (this controls LLM cost). Deployment is a separate one-time charge
LLM tiers	Small model (Haiku-class) for Clarifier and Router. Large model (Sonnet-class) for Planner, Coder, Editor, Fixer. Prompt caching. Per-run cap of 300k tokens
Observability	Langfuse (or LangSmith) traces plus a runs table with tokens, cost and success

3. Scope

In the MVP

- Voice and text chat, transcript confirmation, clarifying questions, requirements summary and confirmation
- Generation of a full vite-react-tailwind project, sandbox build and test, automatic fix loop (max 3)
- Edit flow that touches only the affected files
- Version history on GitHub, rollback, preview URL
- Human deployment workflow: request, assignment, dedicated chat, quote, mock payment, deployment, confirmation
- Ledger with platform commission, developer balances and a monthly payout report
- Subscription plans with quotas

Stretch (only if on schedule)

- static-html profile
- Vision-based screenshot judge in the Verifier
- Urdu-language Clarifier prompts

Out of scope (future work)

- Next.js SSR and generated apps with databases and auth
- Automatic deployment, custom domains, SSL automation
- Real money movement and legal/regulatory work around holding user funds

- Production multi-tenant hardening

4. How it works end to end

1. **Input.** The user types or records audio. Audio goes to S3 and is transcribed by ASR. The user confirms the transcript.
2. **Quota check.** The plan and token limits are checked before any LLM work.
3. **Route.** The Router classifies the message: question, small edit, or new/major change.
4. **Create path.** The Clarifier asks 3 to 5 questions per turn (options where possible, 'you decide' always allowed and stored as a decision) until all requirement fields are filled. A summary is shown; the user confirms or sends it back for changes. Requirements vN are saved.
5. **Plan and code.** The Planner produces a stack-aware file plan. The Coder writes files one at a time.
6. **Edit path.** The Locator uses the file manifest, vector hits and import graph to find 1 to 5 files. If the request changes requirements (new feature, stack change) it goes back to the Clarifier. Otherwise the Editor makes diffs for 1 to 3 files. Ambiguous edits get one short question.
7. **Build and test.** The sandbox runs npm ci, tsc, lint, build, start and Playwright smoke tests. Failures go to the Fixer (max 3 retries). A failed Verifier check also goes to the Fixer.
8. **Failure path.** After 3 failed retries the attempt stays on a draft branch, main is untouched, and the user gets a clear message.
9. **Pass path.** The Verifier checks DOM and screenshots. The Committer commits to main and tags vN. Postgres, pgvector (changed files only) and S3 are updated. The user gets a preview URL.
10. **Feedback loop.** If the user is not satisfied, the request returns to the Router. If satisfied and no deployment is wanted, the project stays editable.
11. **Deployment (human).** A ticket is created, an admin assigns a developer, a dedicated chat opens with repo access, the developer quotes a price, the user pays, the developer deploys, the user confirms the live site.
12. **Money.** The ledger records the platform commission and the developer balance. A monthly report lists what each developer is owed.

5. Four members, four end-to-end slices

The work is split into four vertical slices, one per member. Each member owns their slice end to end: backend, agents, UI, tests and evals. Nobody is only a backend or frontend person, and the effort is about 25% each.

Member	Slice (owned end to end)	What they build	What they prove
Member 1	Conversation and requirements	Voice recording, ASR and transcript confirmation; chat UI with streaming; Router; Clarifier; question-answer path; requirements and decisions storage; summary and confirm; plans and quota checks	Clarifier reaches ready in 3 turns or fewer on 20 personas; ASR error rate on 30 clips
Member 2	Generation engine	Planner, Coder, Fixer; sandbox runner; profile scaffolds with cached dependencies; build-log view in the UI; Plan and BuildResult schemas	Build success on 30 scenarios
Member 3	Edit and versioning	Locator, Editor; file manifest and embeddings; GitHub Committer (MCP); versions table; version history and rollback UI	Edit accuracy on 30 tasks; rollback restores exact files
Member 4	Delivery and business	Verifier (Playwright, screenshots); preview URL; auth and roles; deployment tickets; admin dashboard; dedicated chat; quotes; mock payments; ledger; payout report	Full deployment demo; ledger balances on every payment

Shared foundations, one each (weeks 1 to 5). Member 1: LangGraph skeleton, state and checkpointer. Member 2: CI and the sandbox base image. Member 3: Postgres and pgvector schema, S3, GitHub organisation. Member 4: FastAPI and web app shell, hosting,

Langfuse tracing.

Contracts between slices (agreed in week 1). Kept in one contracts folder as Pydantic schemas: Requirements (Member 1 to 2), Plan and BuildResult (Member 2 to 3 and 4), FileEdit (Member 3 to 2), Version record (Member 3 to 4), DeploymentRequest (Member 4). A contract changes only by a pull request reviewed by the other three.

Keeping the load equal. Members 1 and 2 carry the hardest agents, so Member 4 carries the widest surface (Verifier plus the whole deployment flow) and Member 3 carries versioning and the GitHub integration. Review the split at the end of M1 and move tasks if one member is clearly ahead or behind.

Review and cover rule. Pull requests are reviewed by the next member in a ring (1 reviews 2, 2 reviews 3, 3 reviews 4, 4 reviews 1), so each member knows one other slice well enough to demo it if its owner is absent. Everyone demos their own slice every Friday.

6. Data model

Core

- projects: id, owner, name, stack_profile, current_version, current_req_version, github_repo
- messages: id, project_id, role, content, audio_s3_key, kind, created_at
- requirements: id, project_id, n, data (jsonb), diff, message_id, confirmed_at
- versions: id, project_id, n, parent_n, requirements_n, trigger_message_id, commit_sha, summary, status
- files: version_id, path, summary, exports, imports, embedding (manifest and search only, contents live in Git)
- decisions: project_id, topic, choice, reason, source, message_id
- runs: version_id, agent, tokens_in, tokens_out, cost, ok, trace_id

Accounts and plans

- users (role: user, developer, admin), plans, subscriptions, usage_counters

Marketplace

- deployment_requests: project_id, version_id, status, assigned_developer, chat_room
- quotes: request_id, amount, status
- payments: quote_id, amount, gateway_ref, status (held, released, refunded)

- ledger_entries: account (user, platform, developer), debit, credit, ref, created_at. Every payment must balance
- payout_periods and payouts: month, developer, total, status

7. Timeline (16 weeks from kickoff)

Milestone	Weeks	Goals	Exit criteria
M0 Align and de-risk	1-2	Repo, board and CI set up. Four spikes, one per member: Member 1 runs Whisper on 20 to 30 Urdu/English clips; Member 2 runs a build plus Playwright at 2 GB RAM and one Coder run from a handwritten plan; Member 3 has the Committer create a repo and push files via GitHub MCP; Member 4 serves a static build at a preview URL with a Playwright screenshot. The contracts folder is agreed	All four spikes pass or have a named fallback. Contracts merged. Open questions (section 14) decided
M1 Build engine	3-5	Each member builds a first version of their slice against the contracts. Member 1: LangGraph skeleton, chat UI with streaming, Router (rules). Member 2: Planner, Coder, Fixer, sandbox runner, vite-react-tailwind scaffold. Member 3: Postgres and pgvector schema, S3, manifest builder, GitHub organisation. Member 4: auth and roles, app shell, Verifier, Langfuse	10 scripted prompts produce a site that builds in the sandbox, from an API call with no UI needed. Each slice has tests and a short demo

Milestone	Weeks	Goals	Exit criteria
M2 Create flow	6-8	Member 1: Clarifier with interrupt and checkpointer, requirements schema, summary and confirm, voice recording and ASR with transcript confirmation. Member 2: build-log view, Fixer tuning. Member 3: Committer and versions table, one repo per project. Member 4: preview URL, screenshots to S3	A user chats (typed or spoken), confirms requirements, and gets a verified site committed to GitHub as v1 with a preview
M3 Edit and versions	9-11	Member 3: Locator, Editor, embeddings, rollback and history UI. Member 1: question path, plans and quota checks. Member 2: eval harness for 30 build scenarios. Member 4: deployment tickets and admin dashboard skeleton	Edits touch only expected files. Rollback restores exact files. First full eval report
M4 Human deployment and money	12-14	Member 4: dedicated chat, quotes, mock payments, ledger, payout report. Member 1: subscription plans and quotas end to end. Member 2: cost caps, timeouts and retry tuning. Member 3: 30 edit-task evals and regression checks	Full deployment story works in the demo, ledger balances, quotas enforced

Milestone	Weeks	Goals	Exit criteria
M5 Harden and ship	15-16	Member 1: Urdu ASR evaluation. Member 2: static-html profile (stretch). Member 3: version history polish. Member 4: error states and admin polish. Everyone: final eval run, documentation, demo rehearsals and a recorded backup demo	All acceptance criteria in section 8 measured and reported

Cut line. If the team falls behind, cut in this order: static-html profile, vision judge, subscription quotas (use hardcoded limits), WebSocket chat (use polling), payout report formatting (keep the ledger). Do not cut: sandbox verification, version history, the human deployment flow, voice input.

Rhythm. Monday planning (30 min), Friday demo of whatever works end to end, and every milestone ends with a recorded demo. Log LLM spend every week.

8. Acceptance criteria

Area	Target
Build success	At least 85% of 30 scenarios build and pass smoke tests within 3 fixes (stretch: 95%)
Clarifier	Reaches ready in 3 turns or fewer on 20 scripted personas and asks only missing questions
Edits	Locator picks the right files in 90% or more of 30 edit tasks. Diffs touch only expected files. No regressions in old content
Versioning	Every version links to its requirements version and trigger message. Rollback restores exact files
ASR	Word error rate measured on 30 Urdu/English clips and reported. Transcript confirmation always shown

Area	Target
Human deployment	Request to payout report works end to end in the demo
Ledger	Debits equal credits for every payment, refund and payout
Cost	No run exceeds 300k tokens. Average cost per new site is logged and stays under $3
Security	Sandbox has no secrets. GitHub token is scoped to the platform org. No keys in the repo

9. Budget

Prices were checked against vendor pages in Oct 2026. Re-check before buying anything.

Item	Estimate (USD)
LLM API for development and 3 to 4 full eval runs	150 to 400
Sandbox (E2B Hobby includes $100 one-time credits; a 5 minute run costs about $0.011)	0
ASR (Whisper API about $0.006 per minute, or self-hosted)	under 10
Backend, Postgres, small VPS or free tiers	0 to 60
GitHub organisation, Langfuse free tier, mock payments	0
Domain (optional)	10 to 15
Total	**about 170 to 500 (roughly 48k to 140k PKR at about 280 PKR per USD)**

Unit costs after launch. New site about $1.5 to $2.5 (including fix retries). Edit about $0.05 to $0.15. Sandbox run about $0.01. Voice under $0.01 per minute. This is why the free plan must be capped.

Ways to reduce cost. Haiku/Sonnet split, prompt caching, per-run token caps, one file at a time for the Coder, and re-embedding only changed files. Note that sources disagree on current Sonnet-class pricing ($3/$15 vs $2/$10 per million tokens), so confirm on the official pricing page.

10. Risks and mitigations

Risk	Mitigation
Build pass rate below target	Pinned scaffolds, only 2 profiles, Fixer sees only failing files, weekly evals
Runaway Fixer loops or token cost	Max 3 retries, global step limit, 300k token cap per run, timeouts
Sandbox limits (RAM, 1 hour sessions on Hobby)	Week 1 test, local Docker as fallback
Wrong file located for an edit	Editor can request more files. tsc and build catch cross-file breakage
ASR errors on mixed-language speech	Early test, confirmation step, text fallback
Preview infrastructure is complex	Static output only
Payments and legal scope creep	Ledger and mock gateway only. Real money is future work
GitHub MCP misuse, rate limits or token leak	Only the Committer node calls it, GitHub App or fine-grained token, one private repo per project in a platform org
Team falls behind	The cut line in section 7
Live demo failure from LLM variability	Seeded scenarios, cached runs, recorded backup demo

11. Working agreements

- main is protected. Work happens on feature branches. Every pull request gets one review from another member.

- Trello columns: Backlog, This week, In progress, In review, Done. Slack channels: general, dev, demos.
- Definition of done: merged, tested, demoed, and documented in the repo README or docs folder.
- Every agent boundary uses a Pydantic schema. A schema failure triggers a retry, never silent acceptance.
- Prompts live in the repo and are versioned. Every LLM call is traced.
- Secrets only in environment variables or a secret manager.

12. Final demo script (about 8 minutes)

1. A shop owner speaks a request in Urdu/English. The transcript is shown and confirmed.
2. The Clarifier asks a few option-style questions. The owner answers, including one 'you decide'.
3. The summary is confirmed and requirements v1 are saved.
4. The build runs live. Show sandbox logs, one automatic fix, and the verified result.
5. The preview opens. Show the GitHub repo with commit and tag v1.
6. The owner says 'make the header sticky'. Show only the affected file changing, and tag v2.
7. Roll back to v1 to show version control working.
8. The owner requests deployment. Show the ticket, assignment and dedicated chat.
9. The developer quotes, the owner pays (mock gateway), the developer marks the site deployed, the owner confirms.
10. Open the admin ledger and the monthly payout report showing commission and developer balance.

13. Fix in the proposal deck before submission

- Rewrite the story to match this plan: real code generation, GitHub versioning, human-assisted deployment (the deck still says template injection, zero human intervention and auto-deploy).
- The cost chart shows 250 PKR for the software FYP. Replace it with the budget in section 9.

- Severity 9.0, TAM 90 and Itch 92.4 have no source. Replace them with results from a short survey of 15 to 20 shop owners, or remove them.
- Reduce the solar tracker comparison to one slide.
- Confirm the FAST/UET/PU rubric and Trello/Slack claims against the supervisor's handbook.

14. Decisions needed this week

Decision	Options	Owner
Who takes which slice	Pick by interest and skills, keeping the load near 25% each (section 5)	All
Sandbox	E2B Hobby vs local Docker (decide after the RAM spike)	Member 2
Commission rate	A fixed percentage, for example 15 to 20%	Member 4, with supervisor
Payment gateway for future work	Safepay or another local gateway	Member 4
Hosting for the platform	Free tiers vs one small VPS	Member 4
Preview hosting	Sandbox URL vs static host	Member 4
Plan limits	Quotas for Free and Pro	Member 1
Supervisor sign-off on scope and cut line	Present sections 3 and 7	All